

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ 3

по дисциплине «Методы верификации»

Выполнил:
студент группы № 932223
Захаров А.А.

Проверил:
Преподаватель
Лапутенко А. В.

Задача об обедающих философах. Обнаружение ситуации общего голодания (взаимная блокировка процессов)



Философы – процессы

Вилки – общие ресурсы

Блокировка процессов, конкурирующих за разделяемые ресурсы

- образное описание проблемы, возникающие при совместном использовании ресурсов несколькими параллельными процессами
- за круглым столом сидят несколько философов, каждый из которых проводит время в размышлениях, изредка прерываясь на еду
- на столе стоит блюдо со спагетти и вилки, по одной между каждыми двумя философами
- чтобы поесть, каждый философ должен использовать две вилки, которые лежат непосредственно слева и справа от него
- философ берет вилки, если они не заняты, ест спагетти, после чего возвращает вилки на место
- если правая либо левая вилка занята соседом, философ просто ждет ее освобождения

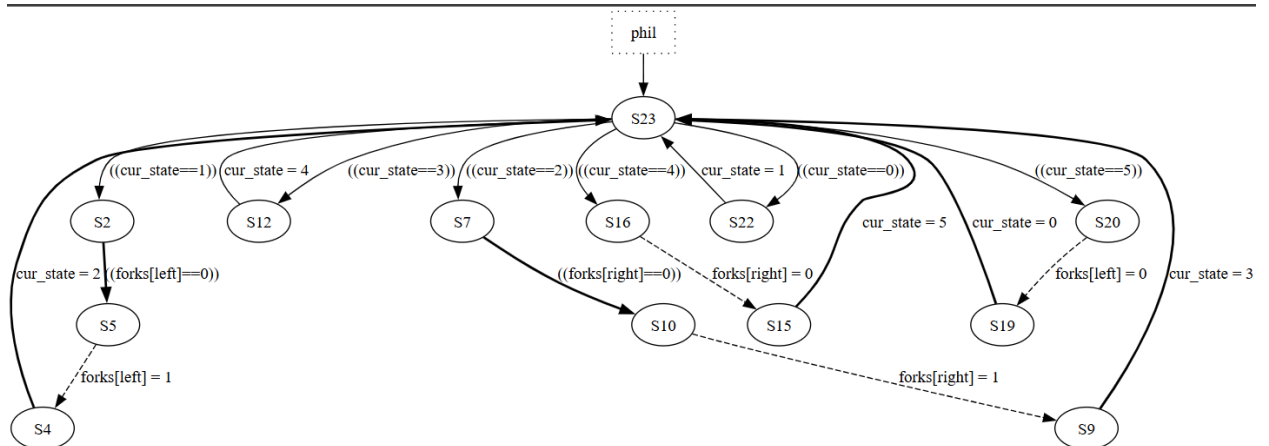
Реализация с возможной блокировкой (deadlock):

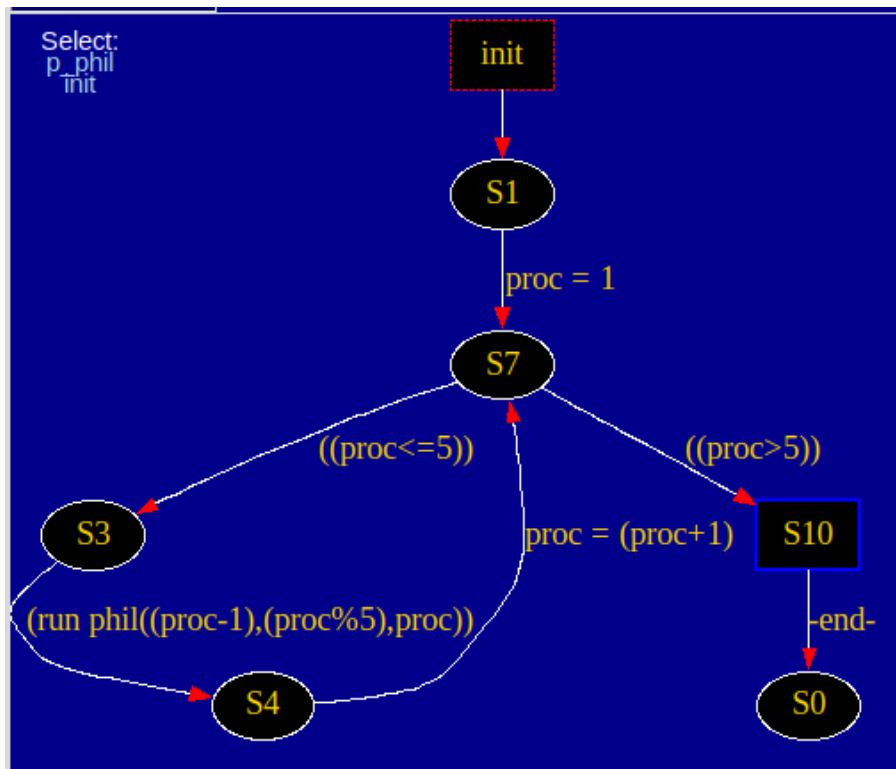
```

#define N 5
bit forks[N];
proctype phil (chan left, right; byte mn)
{
    byte cur_state = 0;
    do
        :: (cur_state == 1) -> (forks[left] == 0) ->
            atomic { forks[left] = 1; cur_state = 2 }
        :: (cur_state == 2) -> (forks[right] == 0) ->
            atomic { forks[right] = 1; cur_state = 3 }
        :: (cur_state == 3) -> cur_state = 4;
        :: (cur_state == 4) -> atomic { forks[right] = 0; cur_state = 5 }
        :: (cur_state == 5) -> atomic { forks[left] = 0; cur_state = 0 }
        :: (cur_state == 0) -> cur_state = 1;
    od }

init
{
    byte proc;
    proc = 1;
    do
        :: proc <= N ->
            run phil (proc-1, (proc%N), proc);    proc++;
        :: proc > N -> break ;
    od
}

```





(Spin Version 6.5.2 -- 6 December 2019)

Warning: Search not completed
+ Partial Order Reduction

Full statespace search for:

never claim - (not selected)
assertion violations +
cycle checks - (disabled by -DSAFETY)
invalid end states +

State-vector 64 byte, depth reached 6570, errors: 1

6571 states, stored

2564 states, matched

9135 transitions (= stored+matched)

0 atomic steps

hash conflicts: 0 (resolved)

Stats on memory usage (in Megabytes):

0.577 equivalent memory usage for states (stored*(State-vector + overhead))

0.777 actual memory usage for states

128.000 memory used for hash table (-w24)

0.534 memory used for DFS stack (-m10000)

129.218 total actual memory usage

pan: elapsed time 1.37 seconds

To replay the error-trail, goto Simulate/Replay and select "Run"

Ошибка тупика (deadlock) на 6570-м (номер шага может отличаться ввиду недетерминизма блоков do od шаге

Реализация с официантом-семафором

```
#define N 5

mtype = { msgtype, REQUEST, GRANT, DONE };

chan c[N] = [0] of { mtype };
chan d[N] = [0] of { mtype };
chan w[N] = [0] of { mtype };

proctype phil(chan left, right, waiter; byte mn) {
    byte cur_state = 0;
    printf("MSC: phil # %d \n", mn);
    do
        :: (cur_state == 1) ->
            waiter ! REQUEST;
            waiter ? GRANT;
            cur_state = 2;
        :: (cur_state == 2) ->
            left ! msgtype;
            cur_state = 3;
        :: (cur_state == 3) ->
            right ! msgtype;
            cur_state = 4;
        :: (cur_state == 4) ->
            cur_state = 5;
        :: (cur_state == 5) ->
            right ! msgtype;
            cur_state = 6;
        :: (cur_state == 6) ->
            left ! msgtype;
            cur_state = 7;
        :: (cur_state == 7) ->
            waiter ! DONE;
            cur_state = 0;
        :: (cur_state == 0) ->
            cur_state = 1;
    od
}
```

```

proctype fork(chan left_phil, right_phil) {
    byte cur_state_fork = 0;
end:
    do
        :: (cur_state_fork == 0) ->
            if
                :: right_phil ? msgtype -> cur_state_fork = 2;
                :: left_phil ? msgtype -> cur_state_fork = 1;
                :: skip;
            fi
        :: (cur_state_fork == 1) ->
            left_phil ? msgtype -> cur_state_fork = 0;
        :: (cur_state_fork == 2) ->
            right_phil ? msgtype -> cur_state_fork = 0;
    od
}

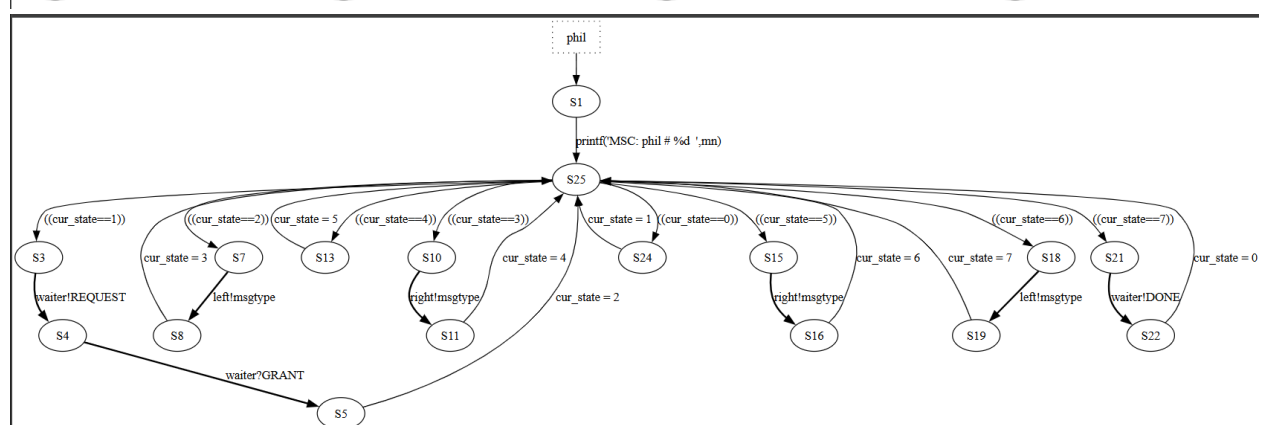
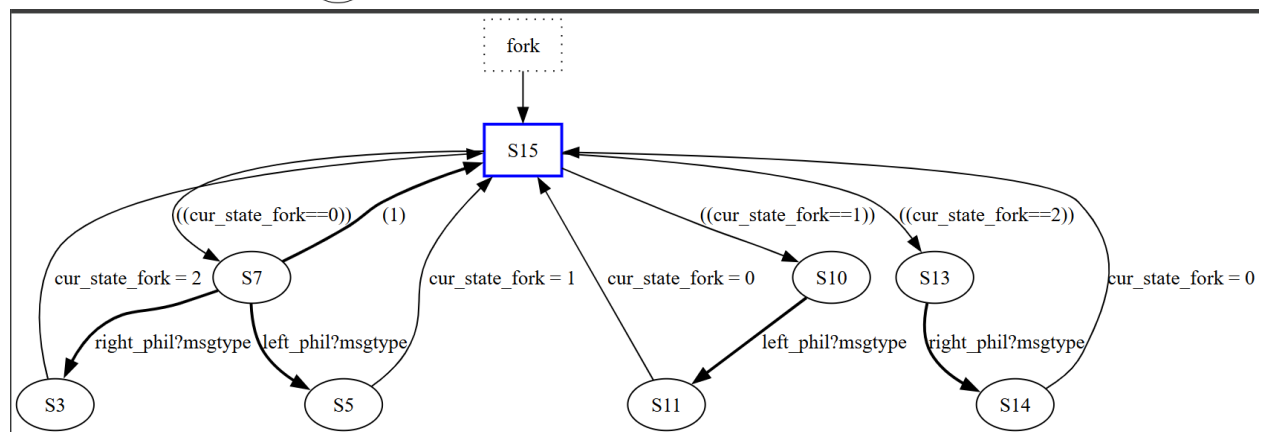
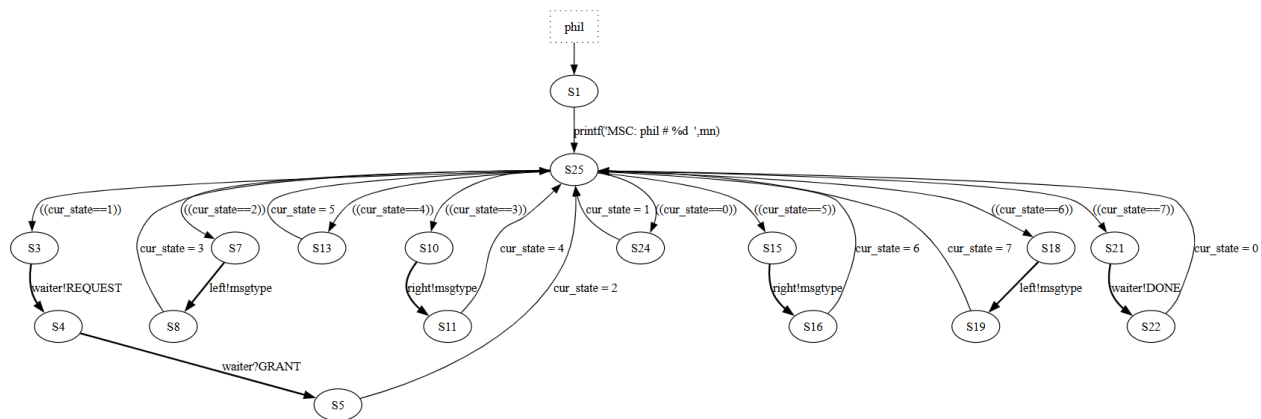
proctype waiter() {
    byte count = 0;
    do
        :: (count < N-1) ->
            if
                :: w[0] ? REQUEST -> w[0] ! GRANT; count++
                :: w[1] ? REQUEST -> w[1] ! GRANT; count++
                :: w[2] ? REQUEST -> w[2] ! GRANT; count++
                :: w[3] ? REQUEST -> w[3] ! GRANT; count++
                :: w[4] ? REQUEST -> w[4] ! GRANT; count++
            fi
        :: (count > 0) ->
            if
                :: w[0] ? DONE -> count--
                :: w[1] ? DONE -> count--
                :: w[2] ? DONE -> count--
                :: w[3] ? DONE -> count--
                :: w[4] ? DONE -> count--
            fi
    od
}

init {
    run phil(d[0], c[0], w[0], 0);
    run phil(d[1], c[1], w[1], 1);
    run phil(d[2], c[2], w[2], 2);
    run phil(d[3], c[3], w[3], 3);
    run phil(d[4], c[4], w[4], 4);

    run fork(c[0], d[1]);
    run fork(c[1], d[2]);
    run fork(c[2], d[3]);
    run fork(c[3], d[4]);
    run fork(c[4], d[0]);

    run waiter();
}

```



Верификация доходит до максимального по номеру шага и заканчивается без ошибки.